

Supporting the Specification of Fairness Requirements

Proposal for a Master Thesis

submitted by

Rahul Thiruvengadam

January 5, 2024

Supervisor: Dr. Qusai Ramadan
Institut für Softwaretechnik

2nd Supervisor: Prof. Dr. Jan Jürjens
Institut für Softwaretechnik

Contents

1	Introduction and Motivation	2
2	Problem Statement	3
3	Research Questions	4
4	Methodology	4
5	Work Packages and Schedule	6
	References	6

1 Introduction and Motivation

In today's rapidly evolving, automated, and data-driven world, software fairness has proved to be a critical concern. These software strive to foster personalized experiences and play a critical role in decision-making, in areas such as healthcare, criminal justice, and recruitment [3]. However, these systems are not devoid of bias and discrimination, as numerous cases have revealed. For example, Amazon's recruiting engine which was designed to streamline the process of reviewing resumes of job applicants and selecting candidates for job interviews, was found to discriminate against women [2]. Another example is Google's translate software, when translating gender-neutral sentences from English to Turkish and then back to English, changes the gender of the profession, indicating gender stereotypes [1]. In the realm of bias and discrimination, the goal is to investigate the various forms of biases that may occur in software systems by identifying their sources and their potential consequences.

The current state of the art mainly focuses on the following facets: Software Fairness Definition, Bias and Discrimination, Fairness verification and validation. However, gaps persist. In the domain of software fairness definitions, researchers Brun and Meliou [1] have underscored the significance of establishing a comprehensive understanding of fairness in the context of software. This encompasses the delineation of fairness criteria and understanding the fundamental role of fairness in addressing bias and discrimination in software systems. Verma and Rubin [9] provide comprehensive definitions and categories of several fairness metrics[7]. Yet, the challenge lies in reconciling these various metrics into unified industry standards. Narayanan's [5] exploration of different fairness definitions highlights the nuanced impact of each definition on outcomes, suggesting the need for consensus-driven definitions to guide practical implementation. Zhang and Harman [10] explore the challenges of achieving fairness in software systems by highlighting the importance of fairness in machine learning software and the need for software engineers to reduce software discrimination. Researchers Varona and Suárez [8] analyze different methods to detect and mitigate biases that may lead to discrimination, and how these outcomes cannot be justified, and the specific trigger cannot be identified.

So far, the inclusion of fairness considerations in software systems is treated as an afterthought rather than being embedded into the system from the onset. Neglecting fairness requirements during the early phases of software development, specifically in the requirements engineering

phase can cause unintentional bias or discrimination, which may have radical ethical, legal, and societal consequences [6]. It is therefore essential for software systems to ensure fairness considerations are an indispensable part of the software development process, beginning from the initial stages.

Specifying fairness requirements is a formidable task due to its complexity and multidisciplinary nature. Misunderstandings due to varying interpretations of what constitutes fairness or incomplete requirement specification may lead to unintentional bias being introduced into the systems [1]. Addressing this challenge is critical to have software free of discrimination or bias.

The first step in supporting the specification of fairness is to develop a conceptual model that identifies and defines the concepts critical to understanding fairness in software development and the relationships between these concepts. This provides consistency and clarity in elucidating fairness concepts, offering a mutual understanding for stakeholders from diverse backgrounds. The current state of the art lacks a conceptual model for specifying fairness. The absence of a conceptual model results in a gap in precisely defining and universally adopting fairness criteria. This gap leads to diverse interpretations and potential discrepancies, hindering the attainment of fairness due to the absence of a standardized framework.

This research aims to fill a critical gap in the field by providing a conceptual model that enhances our understanding of concepts, relationships, and interaction. In the context of software fairness, a conceptual model enhances understanding and clarity surrounding various fairness-related elements and serves as a guide, facilitating fairness requirements classification and risk analysis in software development. Addressing this gap is crucial to bridging the divide between theoretical discussions on fairness and practical implementation, enabling the development of standardized, effective, and bias-free software systems.

In the subsequent sections of this master thesis, a comprehensive exploration of critical elements surrounding fairness in software development unfolds. Section 2 delves into the problem statement and the relevant challenges, outlining the challenges that need to be tackled to support fair systems. Section 3 introduces the research questions guiding this paper; that are aimed to elucidate the pivotal concepts, their relationships in specifying fairness requirements and the evaluation of the model's practical application through a real world case study. Finally, section 4 provides a roadmap of the work packages and schedules, outlining the methodology implemented in addressing the research questions. It also offers a detailed timeline for execution and completion of the defined work packages.

2 Problem Statement

Specifying fairness-related requirements becomes challenging in the absence of a conceptual model. Without a conceptual model, there's a lack of a structured framework outlining fundamental concepts and their relationships. Experts may have to rely on their implicit knowledge to design a fair system however, differences in shared understanding among them might inadvertently lead to biases [4]. For instance, in software development, consider the requirement: "The hiring software should avoid discriminatory candidate selection." This requirement doesn't provide clear criteria as to what actions or parameters define discrimination in candidate selection. Due to the absence of clear definitions or measurable indicators of discrimination, there's no objective way to assess or measure if the software adheres to this requirement. Different interpretations of discrimination could lead to subjective evaluations and challenges in ensuring compliance.

Challenge: Designing a conceptual model for specifying fairness related requirements can be challenging due to its complexity and multidisciplinary nature of integrating ethical, technical, legal, and social elements within software development. The most prominent challenge is the lack of an established glossary that succinctly describes the concepts and relationships pertaining to fairness [1]. This absence of a standardized glossary may introduce ambiguity and misinterpretations thereby, muddling the process of developing a conceptual model. Terms like bias, discrimination and fairness have multiple interpretations and it is essential to have universally accepted definitions to avoid experts unintentionally working from different perspective. For example, “Statistical Parity” and “Demographic Parity” can be used interchangeably to describe equal representation among demographic groups.

3 Research Questions

RQ1: What are the fundamental concepts essential for the specification of fairness-related requirements in software development?

Objective: This research questions identifies the core concepts pertaining to fairness; It aims to delve into the foundational ideas and principles related to specifying fairness-related requirements.

RQ2: How can the relationships between the identified concepts be effectively expressed and structured in a conceptual model for specifying fairness-related requirements in software?

Objective: This research question seeks to understand how the identified concepts from RQ1 can be interconnected and expressed in a conceptual model. It addresses the challenge of creating a comprehensive framework that can guide the specification of fairness-related requirements in software.

RQ3: How can the applicability of the developed conceptual model be evaluated through a real-world case study?

Objective: This research question analyses the practical application of the developed conceptual model by instantiating it with a real-world case study to assess how well the model can support specifying fairness related requirements.

4 Methodology

Phase 1: Identifying Fundamental Concepts (RQ1)

Objective: To identify and compile fundamental concepts essential for specifying fairness-related requirements in software development. This phase focuses on systematically reviewing literature, extracting key concepts, and organizing them into a structured framework.

Input: Academic papers, conference proceedings, journals, and books related to fairness in software, retrieved from databases like IEEE Xplore, ACM Digital Library, Google Scholar, etc.

Process:

- Literature Review - Conduct a systematic review of the literature using Mendeley for citation management.
- Content Analysis - Analyze literature to extract and compile recurring themes, concepts, and definitions related to fairness in software.

- Concept Mapping - Utilize tools like CmapTools or Microsoft Visio to visually map and organize identified concepts and their relationships.

Output: Compilation of key concepts such as bias, discrimination, fairness metrics, and interpretability supported by evidence from the literature.

Phase 2: Conceptual Model Development (RQ2)

Objective: To create a structured conceptual model that effectively represents relationships between identified fairness-related concepts for specifying fairness-related requirements in software development.

Input:

- List of Fundamental Concepts - Obtained from Phase 1, highlighting key elements such as bias, fairness metrics, etc.
- Conceptual Map - A visual representation illustrating relationships between identified concepts from Phase 1.

Process:

- Integration or Modification - Based on the identified concepts, use conceptual modeling software such as Lucidchart, draw.io, or UML tools to integrate or modify existing frameworks/models to accommodate the identified elements.
- Relationship Definition - Define and articulate the connections between each concept to showcase dependencies, influences, and interactions.

Output: A refined and structured conceptual model representing the relationships between identified fairness-related concepts in software development; illustrating the interconnections and hierarchies between the identified concepts.

Phase 3: Model Validation through Case Study (RQ3)

Objective: This phase focuses on the practical validation of the developed conceptual model within a real-world context, allowing for an in-depth assessment of its effectiveness in guiding fairness-related requirements in software development.

Input:

- Developed Conceptual Model - Output from Phase 2, representing structured relationships between fairness-related concepts in software development.
- Selected Real-World Scenario - A specific scenario or software development project chosen for instantiating and evaluating the conceptual model.

Process:

- Application of Model - Implement the developed conceptual model within the context of the selected real-world case study by creating an instance of the conceptual model.
- Evaluation and Assessment - Analyze the effectiveness of the model in the given scenario, assessing its ability to address fairness-related considerations in software development.

- Documentation - Document insights, findings, successes, and limitations observed during the application of the conceptual model in the case study.

Output: Detailed documentation assessing the practical application of the conceptual model in the real-world scenario, highlighting successes, challenges, and any necessary refinements.

5 Work Packages and Schedule

Work Package 1: Literature Review and Conceptual Model Assessment (8 Weeks)

Task 1.1: Identify primary research areas and gather relevant literature.

Task 1.2: Conduct a comprehensive review to identify and analyze preexisting conceptual models related to fairness in software.

Task 1.3: Assess the suitability and adaptability of the preexisting models to your research objectives.

Task 1.4: Begin groundwork for creating a new conceptual model if needed.

Work Package 2: Conceptual Model Development (8 Weeks)

Task 2.1: If using a preexisting model, integrate and modify it according to your research requirements. If creating a new model, define key components and relationships based on literature insights.

Task 2.2: Continuation of integration or creation process, ensuring alignment with research goals.

Task 2.3: Document the development or Integration process of the conceptual model.

Work Package 3: Validation of Conceptual Model (3 Weeks)

Task 3.1: Selection of a real-world case study aligned with either the adapted preexisting model or the new conceptual model.

Task 3.2: Application of the selected model (preexisting or new) to the case study.

Task 3.3: Analyze and document outcomes from the application of the chosen model.

Task 3.4: Reflection and identification of insights gained from the case study application.

Work Package 4: Thesis Write-Up and Documentation (5 Weeks)

Task 4.1: Draft the thesis chapters.

Task 4.2: Detailed write-up of the chapters.

Task 4.3: Compile the thesis document with proper formatting and citations.

Task 4.4: Revisions, proofreading, and finalization of the thesis document.

Task 4.5: Thesis defense

References

- [1] Yuriy Brun and Alexandra Meliou. „Software Fairness“. In: *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2018. Lake Buena Vista, FL, USA: Association for Computing Machinery, 2018, pp. 754–759. ISBN: 9781450355735. DOI: 10.1145/3236024.3264838. URL: <https://doi.org/10.1145/3236024.3264838>.

- [2] Jeffrey Dastin. „Insight - Amazon scraps secret AI recruiting tool that showed bias against women“. en-US. In: *Reuters* (Oct. 2018). URL: <https://www.reuters.com/article/us-amazon-com-jobs-automation-insight-idUSKCN1MK08G/>.
- [3] Elizabeth E. Joh. „Reckless automation in policing“. In: *Social Science Research Network* (Jan. 2022). DOI: 10.2139/ssrn.4009911. URL: <https://doi.org/10.2139/ssrn.4009911>.
- [4] Christian Kästner. „Fairness in machine learning and ML-Enabled products“. en. In: *Medium* (July 2022). URL: <https://ckaestne.medium.com/fairness-in-machine-learning-and-ml-enabled-products-8ee05ed8ffc4>.
- [5] Arvind Narayanan. *Tutorial: 21 fairness definitions and their politics*. Video. Mar. 2018. URL: <https://www.youtube.com/watch?v=jIXIuYdnnyk>.
- [6] Harsha Perera et al. „The Impact of Considering Human Values during Requirements Engineering Activities“. In: *CoRR abs/2111.15293* (2021). arXiv: 2111.15293. URL: <https://arxiv.org/abs/2111.15293>.
- [7] Ezekiel Soremekun et al. *Software Fairness: An Analysis and Survey*. 2022. arXiv: 2205.08809 [cs.SE].
- [8] Daniel Varona and Juan Luis Suárez. „Discrimination, Bias, Fairness, and Trustworthy AI“. In: *Applied Sciences* 12.12 (2022). ISSN: 2076-3417. DOI: 10.3390/app12125826. URL: <https://www.mdpi.com/2076-3417/12/12/5826>.
- [9] Sahil Verma and Julia Rubin. „Fairness Definitions Explained“. In: *Proceedings of the International Workshop on Software Fairness*. FairWare '18. Gothenburg, Sweden: Association for Computing Machinery, 2018, pp. 1–7. ISBN: 9781450357463. DOI: 10.1145/3194770.3194776. URL: <https://doi.org/10.1145/3194770.3194776>.
- [10] Jie M. Zhang and Mark Harman. „"Ignorance and Prejudice" in Software Fairness“. In: *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 2021, pp. 1436–1447. DOI: 10.1109/ICSE43902.2021.00129.